CONTROL

- The central point of this chapter is to use motor feedback via encoders for velocity control and position control of motors.
- We will exemplify this by a stepwise introduction of PID (Proportional, Integral, Derivative) control
- In Chapter 4, we showed how to drive a motor forward or backward and how to change its speed.
- However, because of the lack of feedback, the actual motor speed could not be verified.
- For example, a motor will run faster when free spinning than under load
- In order to control the motor speed we do need feedback from the motor shaft encoders.
- Feedback control is called “closed loop control” (simply called “*control*”), as opposed to “open loop control”, which was discussed in Chapter 4

ON-OFF CONTROL

ON-OFF CONTROL

- The idea behind feedback control is very simple.
  - We have a desired speed, specified by the user or the application program, and we have the current actual speed, measured by the shaft encoders.
  - Measurements and actions can be taken very frequently, for example 100 times per second (EyeBot) or up to 20,000 times per second.
  - The action taken depends on the controller model, several of which are introduced in the following sections.
- However, in principle the action always looks similar like this:
  - ✓ In case desired speed is higher than actual speed:

    Increase motor power by a certain degree.
  - ✓ In case desired speed is lower than actual speed:

    Decrease motor power by a certain degree.

ON-OFF CONTROL

- In the simplest case, power to the motor is either switched on (when the speed is too low) or switched off (when the speed is too high).
- This control law is represented by the formula below, with:

$$R(t) = \begin{cases} K_C & if\ v_{act}(t) < v_{des}(t) \\ 0 & otherwise \end{cases}$$

$R(t) \rightarrow motor\ output\ function$

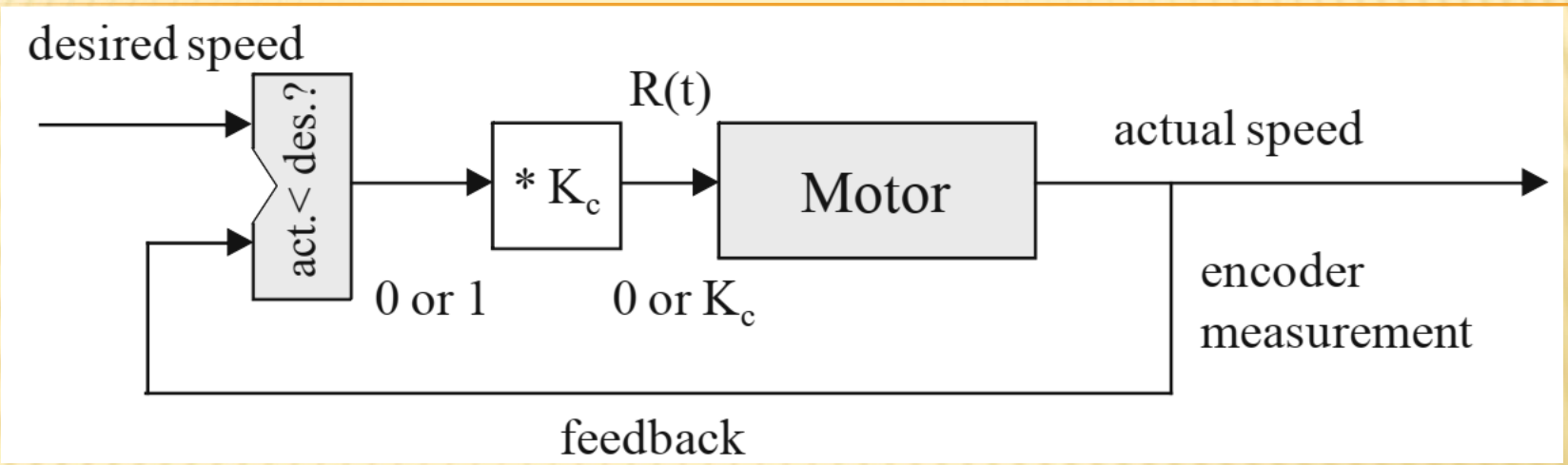
$v_{act}(t) \rightarrow actual\ motor\ speed$

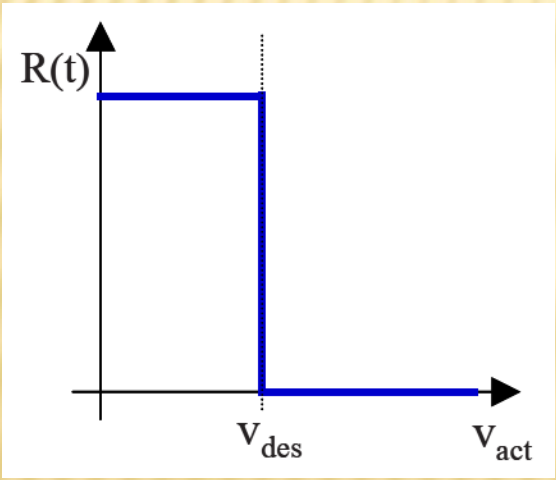
$v_{des}(t) \rightarrow desired\ motor\ speed$

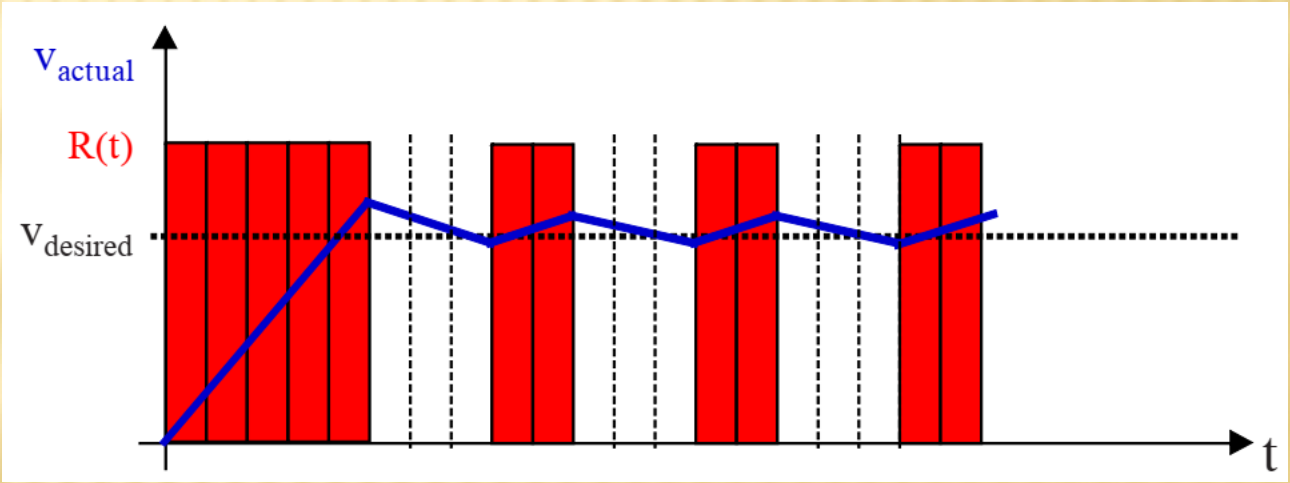
$K_C \rightarrow constant\ control\ value$

- This is the concept of an on-off controller, also known as "piecewise constant controller" or "bang-bang controller"
- The motor input is set to constant value K_C if the measured velocity is too low, otherwise it is set to zero

ON-OFF CONTROL



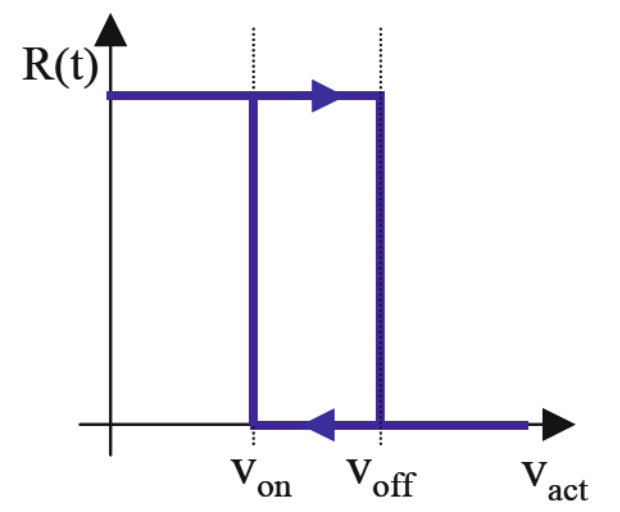


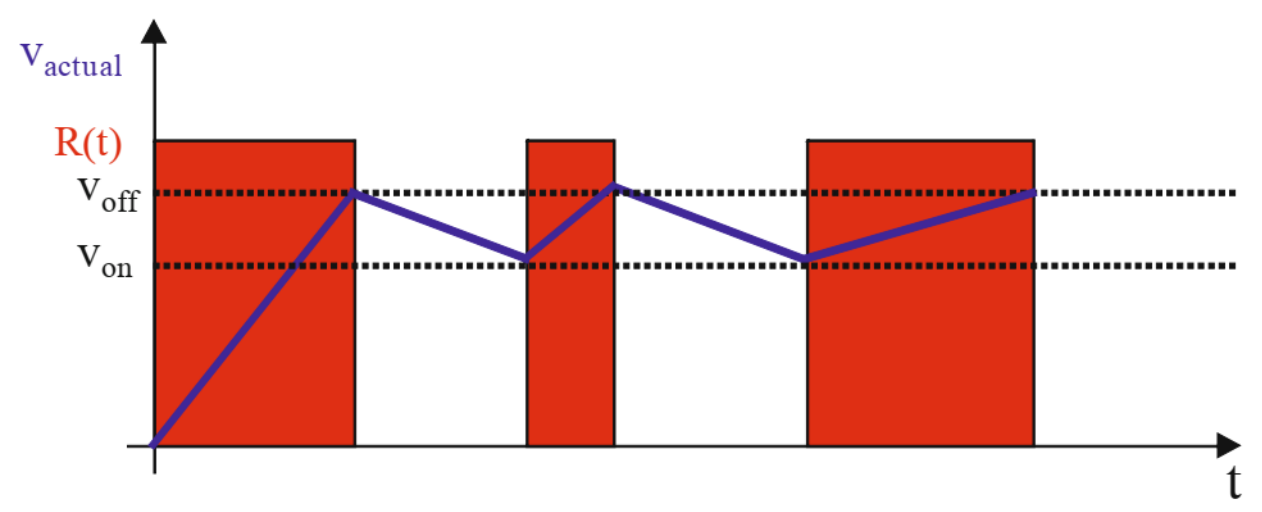


ON-OFF CONTROL WITH HYSTERESIS

- The on-off controller is the simplest possible method of control.
- Many technical systems use it, not limited to controlling a motor.
- Examples are a refrigerator, heater, thermostat, etc.
- Most of these technical systems use a hysteresis band, which consists of two desired values, one for switching on and one for switching off.
- This prevents a too high switching frequency near the desired value, in order to avoid excessive wear

$$R(t+\Delta t) = \begin{cases} K_C & v_{act}(t) < v_{on}(t) \\ 0 & v_{act}(t) > v_{off}(t) \\ R(t) & otherwise \end{cases}$$





PID CONTROLLER

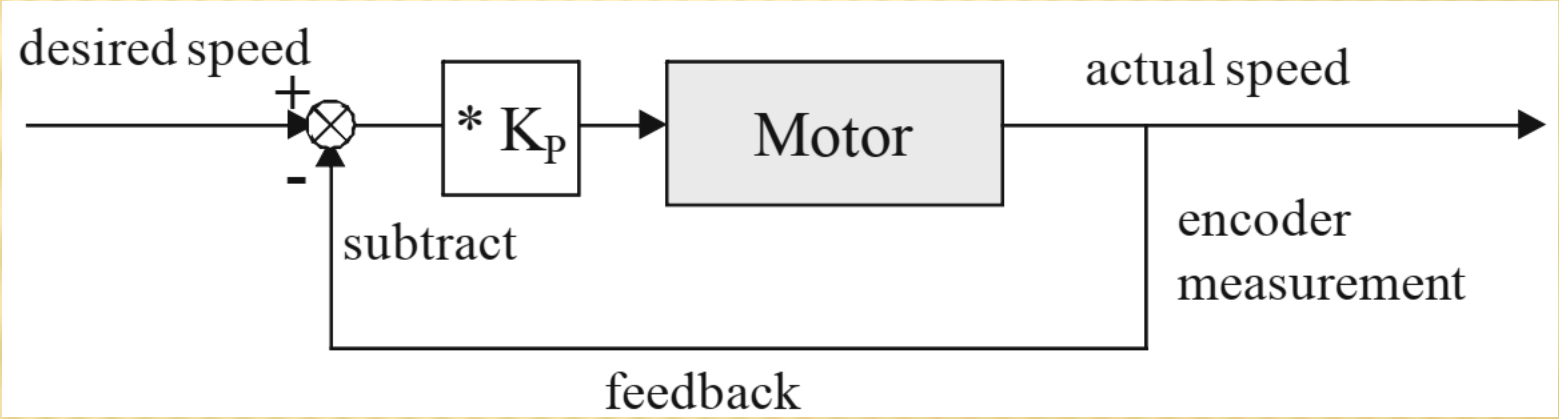
- The simplest method of control is not always the best.
- A more advanced controller and almost industry standard is the PID controller.
- It comprises a proportional, an integral, and a derivative control part.
- The controller parts are introduced in the following sections individually and in combined operation

PROPORTIONAL CONTROLLER

- For many control applications, the abrupt change between a fixed motor control value and zero does not result in a smooth control behavior.
- We can improve this by using a linear or proportional term instead.
- The formula for the proportional controller (P controller) is

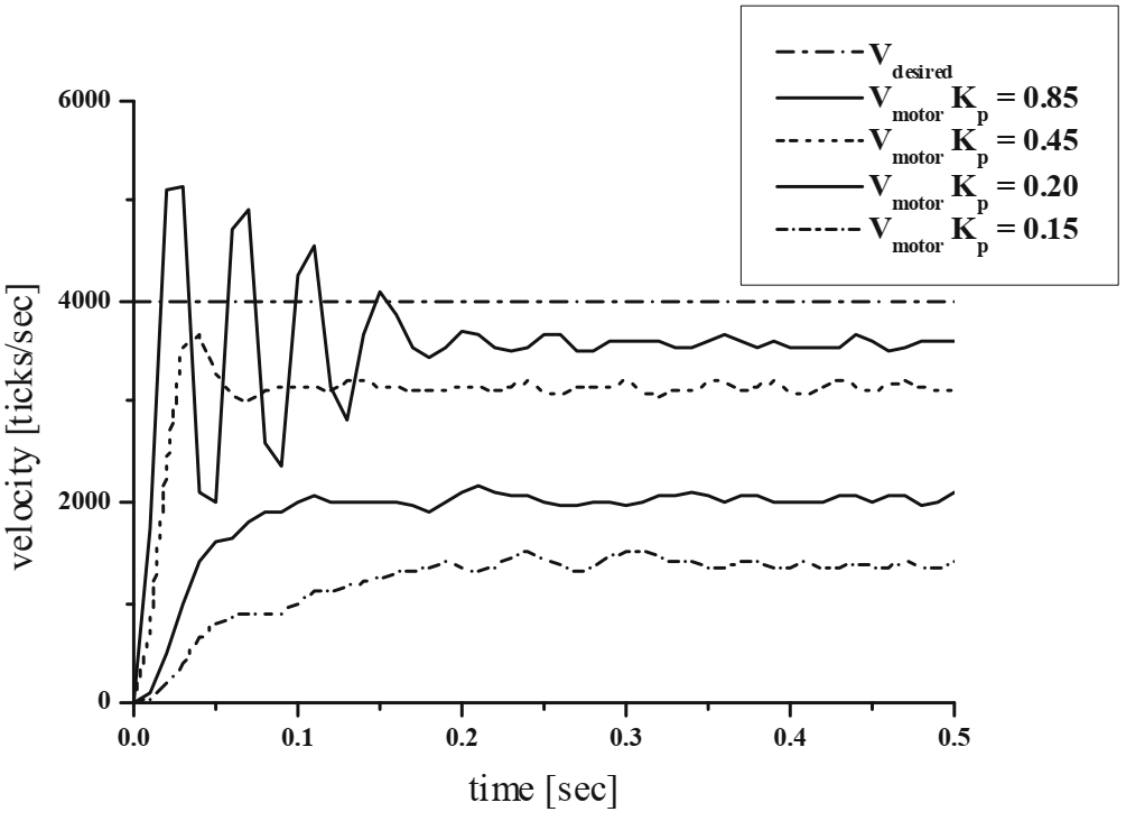
$$R(t) = K_P\left(v_{des}(t) - v_{act}(t)\right)$$

- Difference between the desired and actual speed is called "error function".



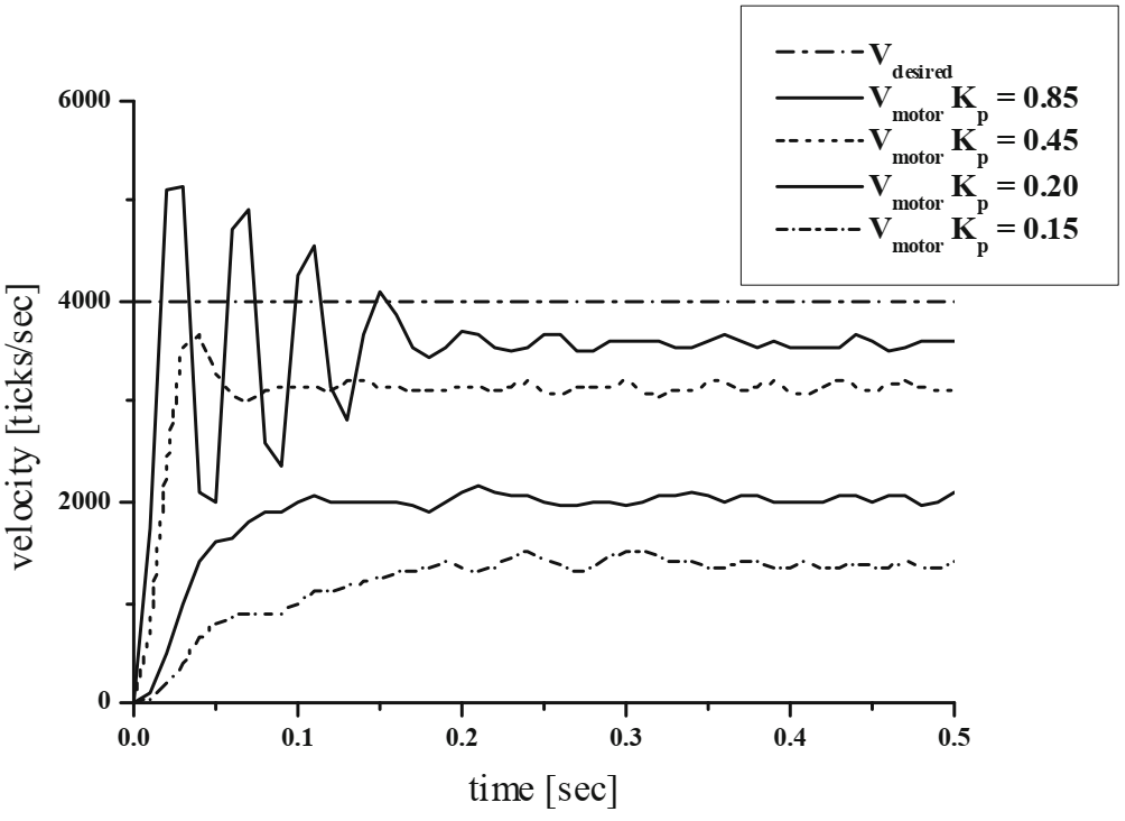
PROPORTIONAL CONTROLLER

- The higher the K_P chosen, the faster the controller responds; however, a too high value will lead to an undesirable oscillating system.
- Therefore it is important to choose a value for K_P that guarantees a fast response but does not lead the control system to overshoot too much or even oscillate



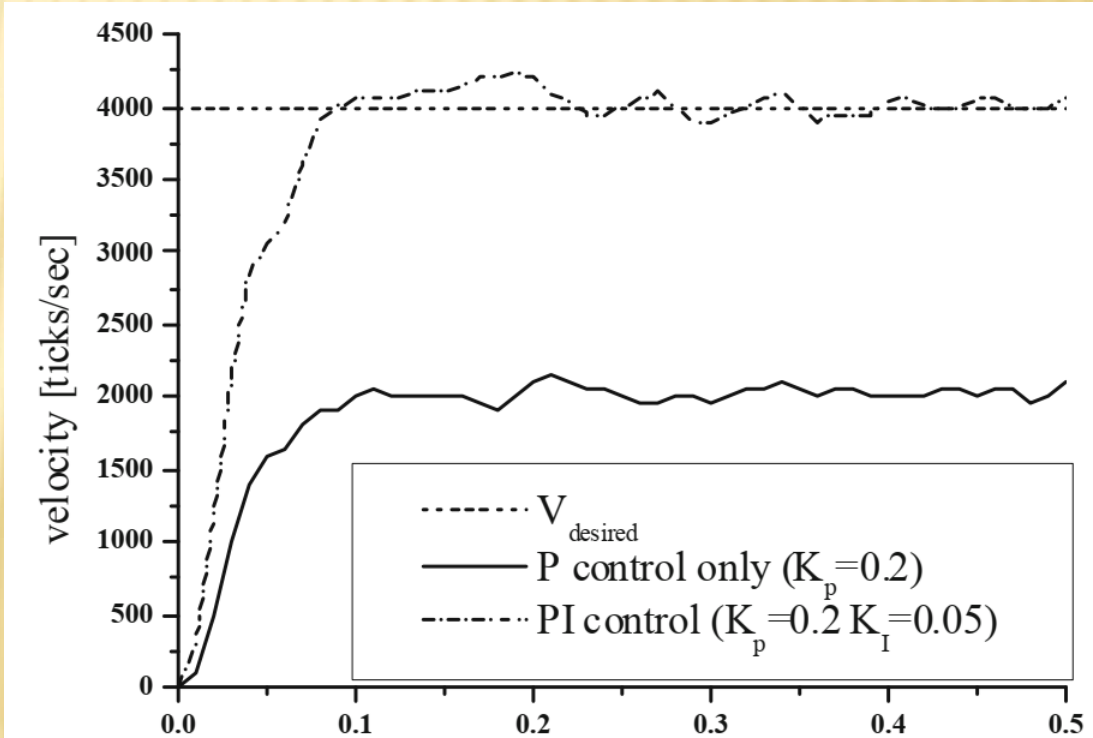
PROPORTIONAL CONTROLLER

- Note that the P controller's equilibrium state is not at the desired velocity.
- If the desired speed is reached exactly, the motor output is reduced to zero, as defined in the P controller formula shown above.
- Therefore, each P controller will keep a certain "steady-state error"
- The higher the gain K_P, the lower the steady-state error.
- However, the system starts to oscillate if the selected gain is too high



INTEGRAL CONTROLLER

- Unlike the P controller, the I controller (integral controller) is rarely used alone, but mostly in combination with the P or PD controller.
- The idea is to reduce the steady-state error of the P controller.
- With an additional integral term, this steady-state error can be reduced to zero, as seen in the measurements in Figure 5.9.
- The equilibrium state is reached somewhat later than with a pure P controller, but the steady-state error has been eliminated



INTEGRAL CONTROLLER

$$R(t) = K_P\left(e(t) + \frac{1}{T_I}\int_0^t e(t)\,\mathrm{dt}\right)$$

$$R(t) = K_P\,e(t) + Q_I\int_0^t e(t)\,\mathrm{dt}$$

$$R_n = K_P e_n + Q_I \Delta t \sum_{i=1}^{n} \frac{e_i - e_{i-1}}{2}$$

$$R_{n-1} = K_P e_{n-1} + Q_I \Delta t \sum_{i=1}^{n-1} \frac{e_i - e_{i-1}}{2}$$

$$R_n = R_{n-1} + K_P(e_n - e_{n-1}) + Q_I \Delta t \frac{(e_n - e_{n-1})}{2}$$

$$\frac{(e_n - e_{n-1})}{2}$$

It is important to *limit the controller output* to the correct value range (for example -100 .. +100 in RoBIOS)

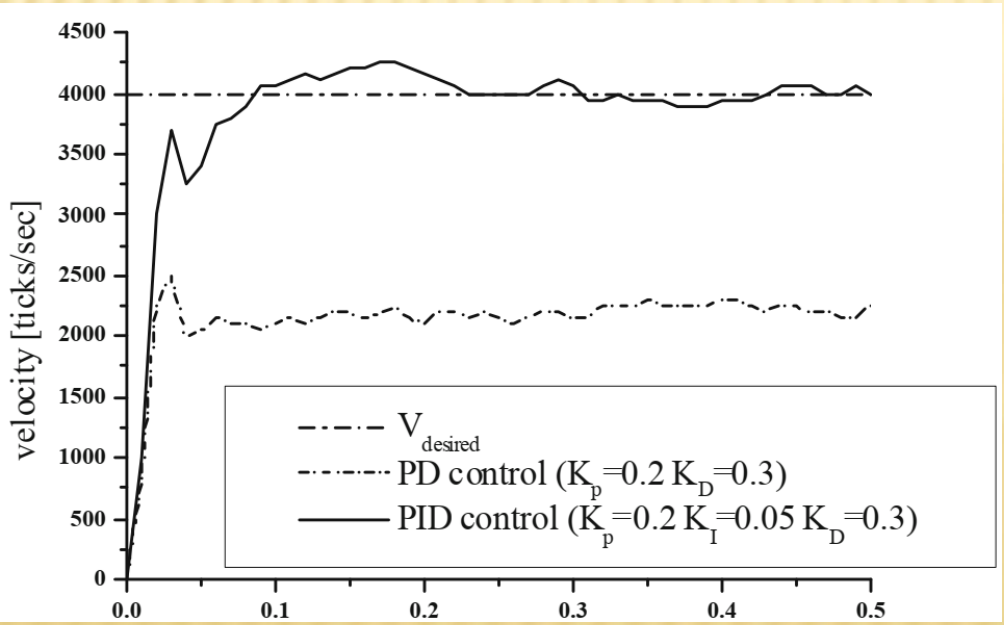
Otherwise, if a desired speed value cannot be reached, both controller output and error values can become very large and invalidate the whole control process [Kasper 2001]

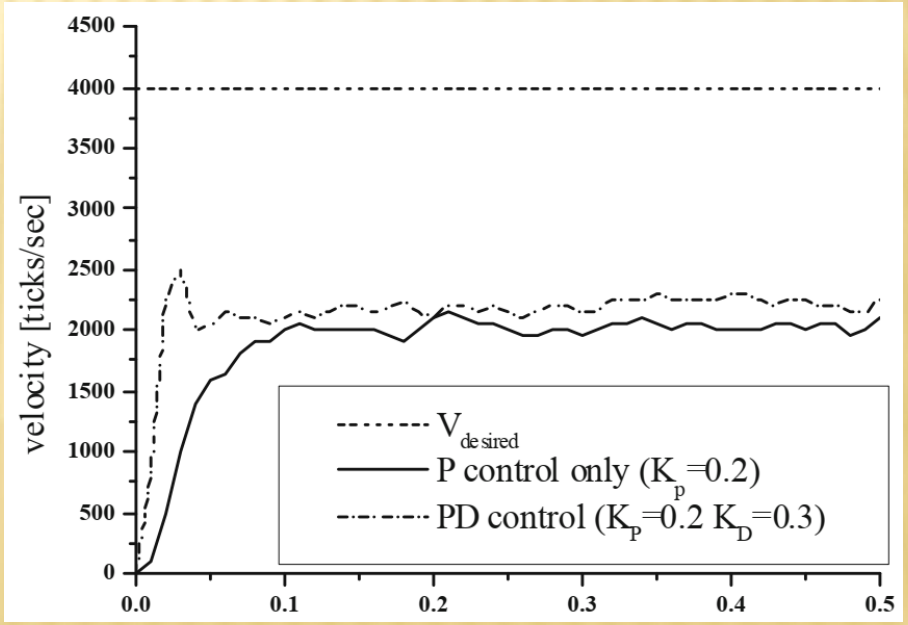
transform the
rror values.
and added for
discretization,

- So we only need to store the previous control value and the previous error value to calculate the PI output in a much simpler formula

DERIVATIVE CONTROLLER

- Similar to the I controller, the D controller (derivative controller) is rarely used by itself, but mostly in combination with the P or PI controller.
- The idea for adding a derivative term is to speed up the P controller's response to a change of input.
- The PD controller reaches equilibrium faster than the P controller, but still has a steady-state error.
- The full **PID** controller combines the advantages of PI and PD. It has a fast response and suffers no steady-state error





DERIVATIVE CONTROLLER

$$R(t) = K_{\mathrm{P}}\left(e(t) + T_{\mathrm{D}}\frac{de(t)}{dt}\right)$$

$$R(t) = K_{\mathrm{P}}\,e(t) + Q_{\mathrm{D}}\frac{de(t)}{dt}$$

$$R_n = K_{\mathrm{P}}e_{\mathrm{n}} + Q_{\mathrm{D}}\frac{e_{\mathrm{n}} - e_{\mathrm{n-1}}}{\Delta t}$$

$$R_{n-1} = K_{\mathrm{P}}e_{\mathrm{n-1}} + Q_{\mathrm{D}}\frac{e_{\mathrm{n-1}} - e_{\mathrm{n-2}}}{\Delta t}$$

$$R_n = R_{n-1} + K_{\mathrm{P}}(e_{\mathrm{n}} - e_{\mathrm{n-1}}) + K_{\mathrm{D}}\frac{(e_{\mathrm{n}} - 2e_{\mathrm{n-1}} - e_{\mathrm{n-2}})}{2}$$

- The formula for the full PID controller is:

$$R_n = R_{n-1} + K_{\mathrm{P}}(e_{\mathrm{n}} - e_{\mathrm{n-1}}) + K_{\mathrm{I}}\frac{(e_{\mathrm{n}} - e_{\mathrm{n-1}})}{2} + K_{\mathrm{D}}\frac{(e_{\mathrm{n}} - 2e_{\mathrm{n-1}} - e_{\mathrm{n-2}})}{2}$$

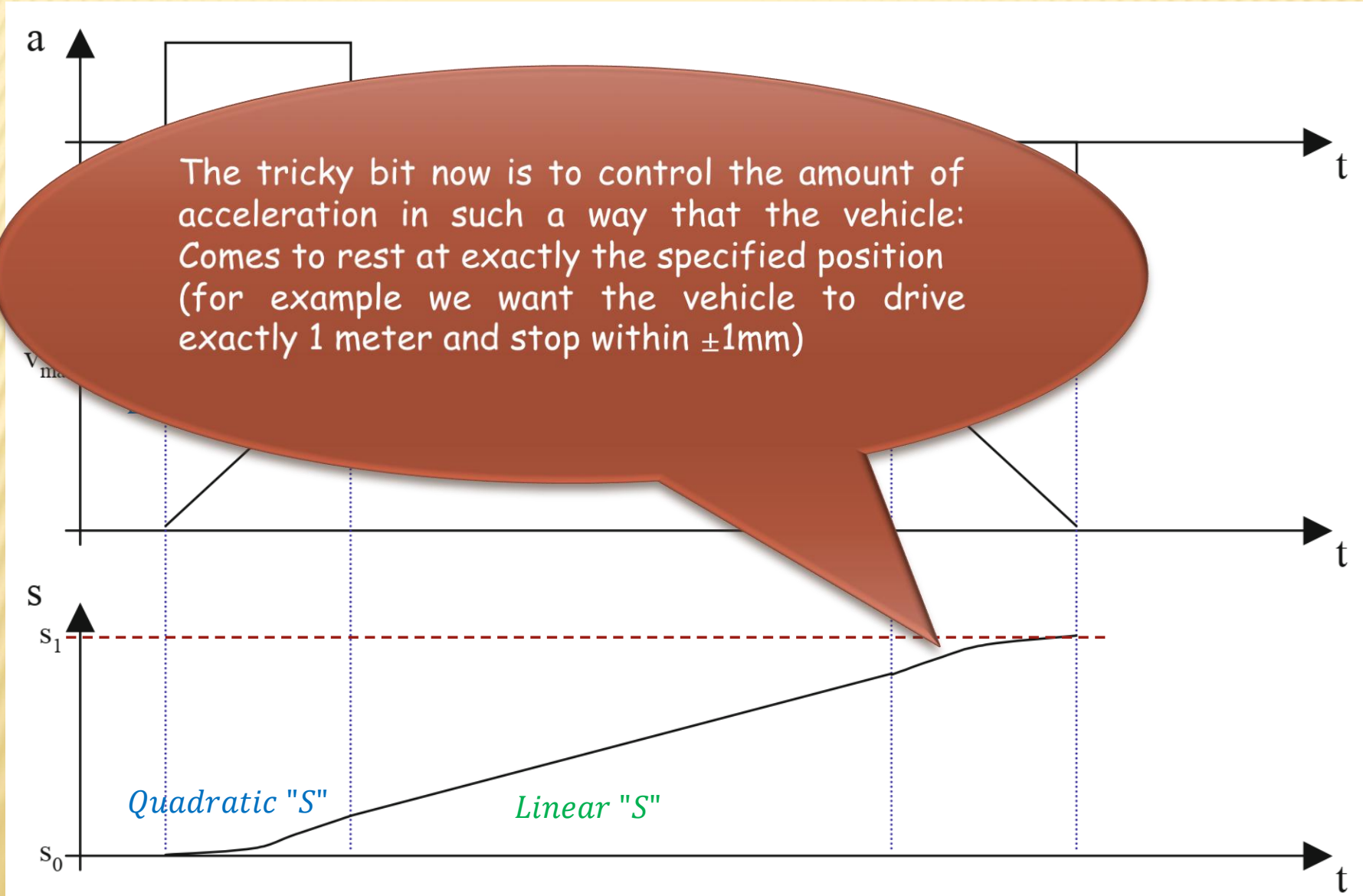
PID PARAMETER TUNING

- The tuning of the three PID parameters K_P, K_I, and K_D is an important issue.
- The following guidelines can be used for experimentally finding suitable values (adapted after [Williams 2006]):
  1. Select a typical operating setting for the desired speed, turn off integral and derivative parts, then increase K_P to maximum or until oscillation occurs.
  2. If system oscillates, divide K_P by 2.
  3. Increase K_D and observe behavior when increasing/decreasing the desired speed by about 5%. Choose a value of K_D which gives a damped response.
  4. Slowly increase K_I until oscillation starts. Then divide K_I by 2 or 3.
  5. Check whether overall controller performance is satisfactorily under typical system conditions.

VELOCITY CONTROL AND POSITION CONTROL

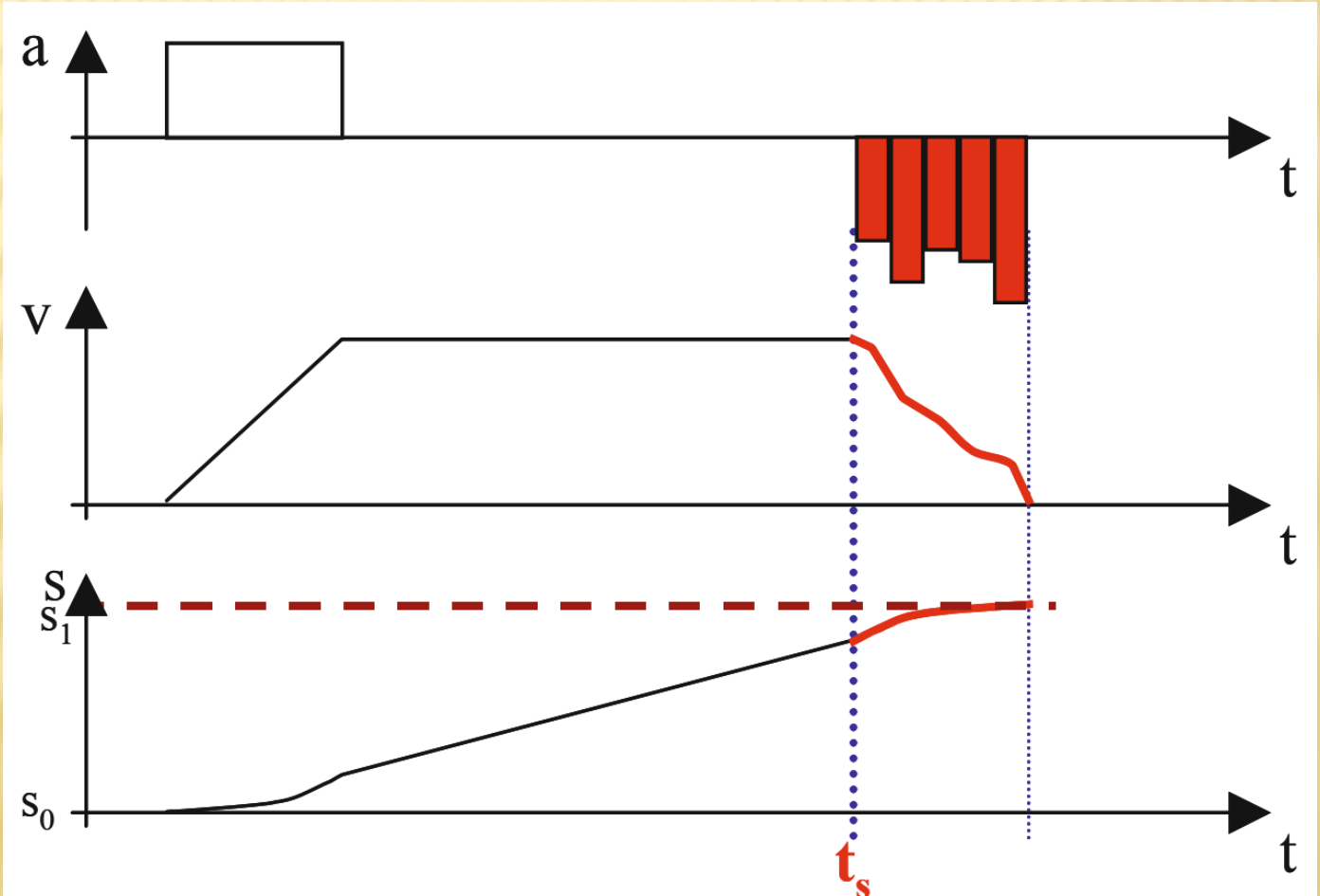
- So far, we are able to control a single motor at a certain speed, but we are not yet able to drive a motor at a given speed for a number of revolutions and then come to a stop at exactly the right motor position.
- The former, maintaining a certain speed, is generally called velocity control, while the latter, reaching a specified position, is called position control.
- Position control requires an additional controller on top of the previously discussed velocity controller.
- The position controller sets the desired velocities in all driving phases, especially during the acceleration and deceleration phases (starting and stopping)
- Let us assume a single motor is driving a robot vehicle that is initially at rest and which we would like to stop at a specified position.
- Figure 5.11 demonstrates the "speed ramp" for the starting phase, constant speed phase, and stopping phase

VELOCITY CONTROL AND POSITION CONTROL



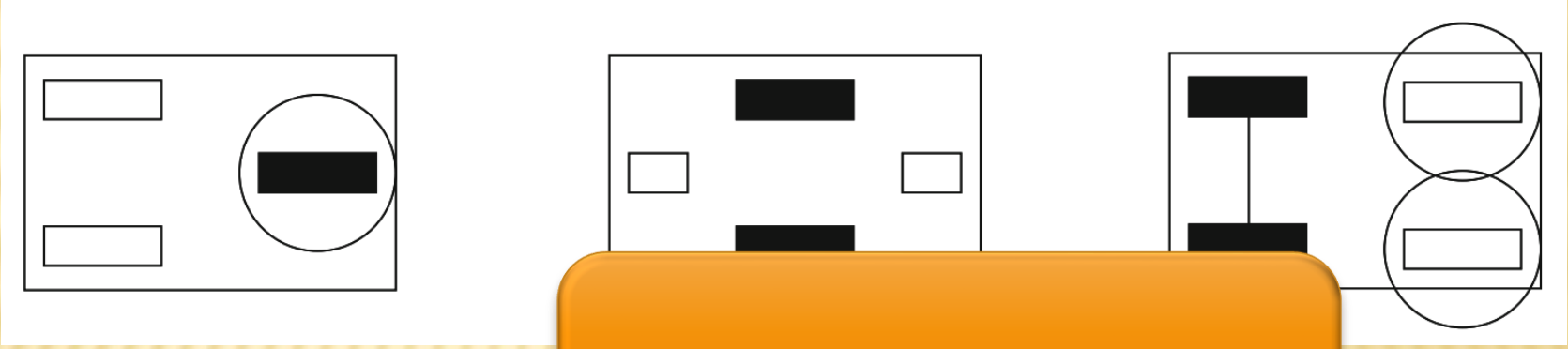
VELOCITY CONTROL AND POSITION CONTROL

- Figure shows a way of achieving this by controlling (continuously updating) the braking acceleration applied.
- This control procedure has to take into account not only the current speed as a feedback value, but also the current position.



MULTIPLE MOTORS – DRIVING STRAIGHT

- Unfortunately, this is still not the last of the motor control problems.
- So far, we have only looked at a single isolated motor with velocity control – and very briefly at position control.
- The way that a robot vehicle is constructed, however, shows us that a single motor is not enough

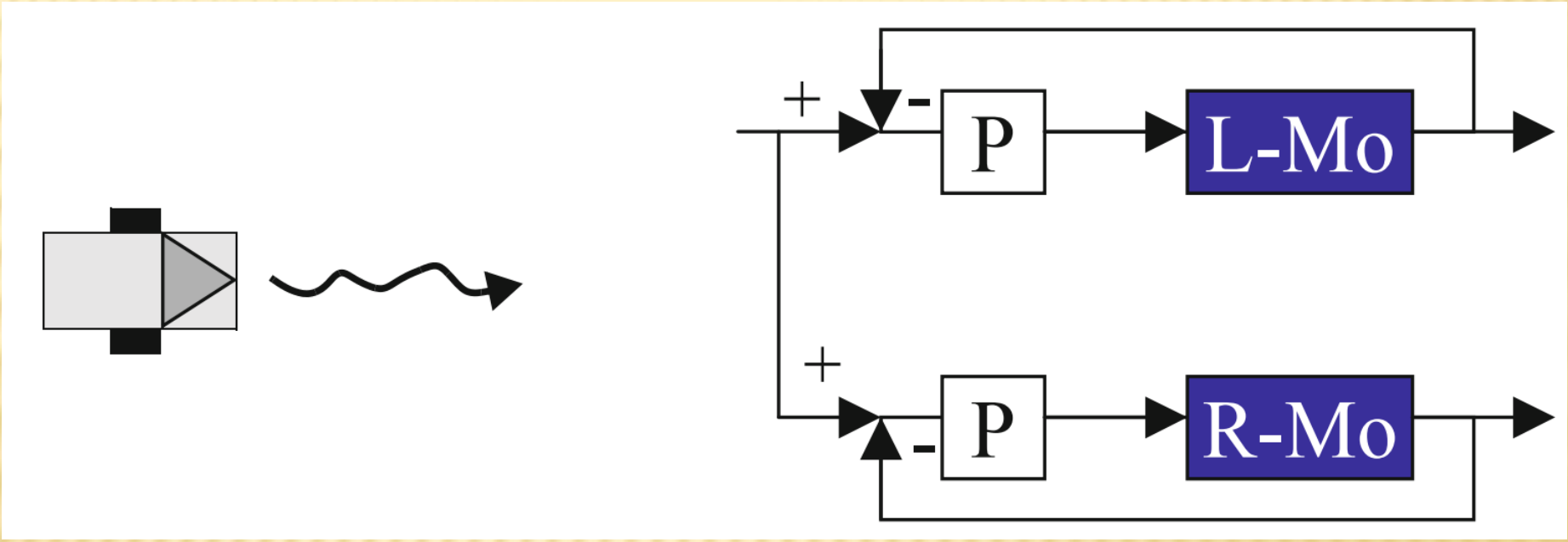


Synchronization of the two motor speeds is required

- In the design on the left o … g and steering functions are separated. I… straight line
- The situation is quite diffe… esign
- Here, one has to constantly *monitor and update* both motor speeds in order to drive straight
- Driving in a circle can be achieved by adding a constant offset to one of the motors.

MULTIPLE MOTORS – DRIVING STRAIGHT

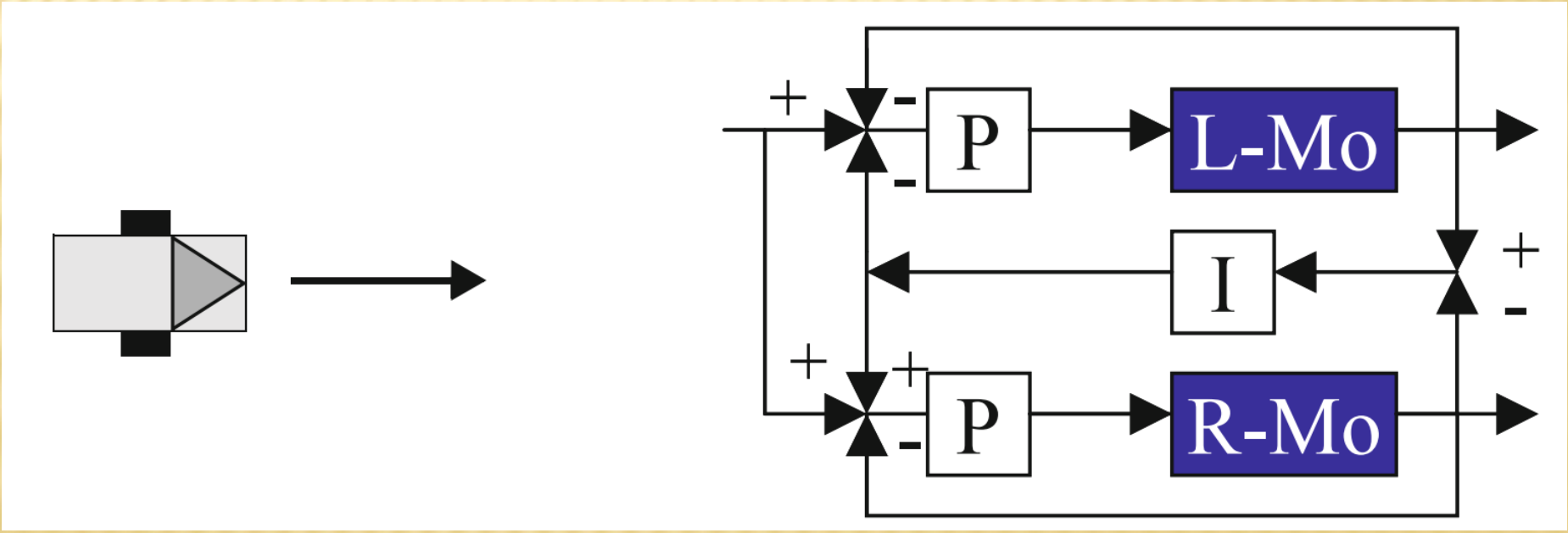
- There are a number of different approaches for driving straight.
- The idea presented in the following is from [Jones, Flynn, Seiger 1999]



- Unfortunately, this design will not produce a nice straight line of driving.
- Although each motor is controlled in itself, there is no control of any slight speed discrepancies between the two motors

MULTIPLE MOTORS – DRIVING STRAIGHT

- An improvement of this control structure is shown in Figure 5.15 as a second try.
- We now also calculate the difference in motor movements (position, not speed) and feed this back to both P controllers via an additional I controller.
- The I controller integrates (or sums up) the differences in position, which will subsequently be eliminated by the two P controllers



MULTIPLE MOTORS – DRIVING STRAIGHT

- For the final version in Figure 5.16 (adapted from [Jones, Flynn, Seiger,1999]), we added another user input with the curve offset.
- With a curve offset of zero. the system behaves exactly like the previous one for driving straight.
- A positive or negative fixed curve offset, however, will let the robot drive in a counter-clockwise or clockwise circle, respectively.
- The radius can be calculated from the curve offset amount.

